AI Product Development Course Description

Catalog Year: 2027 · Foundation Core and All Specialization Tracks

Foundation Core (Required Hours: 360, Credits: 12)	Credits	Hours
Introduction to AI Product Development	1.00	30.00
Product Thinking and Planning	2.00	60.00
Software Systems and Architecture	2.00	60.00
Software Development with AI	3.00	90.00
AI Agents and Automation	4.00	120.00

Design Track (Required Hours: 360, Credits: 12)	Credits	Hours
UX Foundations and Human-Centered Design	2.00	60.00
Visual Design and Prototyping	3.00	90.00
Design Systems	2.00	60.00
User Research and Usability Testing	3.00	90.00
Product Management, Roadmapping, and Agile Practice	2.00	60.00

Web Track (Required Hours: 360, Credits: 12)	Credits	Hours
Modern Front-End, AI-Accelerated	2.00	60.00
React and Component Architecture	3.00	90.00
Back-End and API Engineering	3.00	90.00
Cloud, DevOps and Web Security	2.00	60.00
Capstone and Career Launch	2.00	60.00

iOS Track (Required Hours: 360, Credits: 12)	Credits	Hours
Swift Fundamentals, AI-Accelerated	2.00	60.00
SwiftUI and App Architecture	3.00	90.00
Networking, Persistence and Cloud Backends	3.00	90.00
App Store, DevOps and App Marketing	2.00	60.00
Capstone and Career Launch	2.00	60.00

Foundation Core Courses (Required Hours: 360, Credits: 12)

Foundation Core (Required Hours: 360, Credits: 12)	Credits	Hours

Introduction to AI Product Development **1.00** **30.00**

The Introduction to AI Product Development course orients students to the world of AI-powered software development before any technical work begins. Students get a plain-language introduction to how AI systems work: what they are doing when they generate text, code, or decisions, and where they commonly fall short. The course also maps out what a modern technology company looks like: the roles on a product team and how those roles collaborate. The course closes with every student publishing a simple website built almost entirely with AI assistance.

Course Objectives:

- Explain in plain language how AI systems generate outputs and where they commonly fail.
- Identify the key roles on a software product team and describe how they collaborate.
- Describe how a technology organization is structured and where individual roles fit within it.
- Use AI tools to plan, build, and publish a simple website without prior technical experience.

Product Thinking and Planning **2.00** **60.00**

The Product Thinking and Planning course introduces the product development lifecycle from idea to launch: identifying a real user problem worth solving, gathering and prioritizing requirements, and translating a rough idea into a plan a team can act on. Students learn to write user stories and acceptance criteria and use a basic prioritization framework to decide what to build first and what to leave out.

Course Objectives:

- Describe the stages of a typical product development lifecycle from idea to launch.
- Identify a real user problem and articulate why it is worth solving.
- Write clear user stories and acceptance criteria.
- Gather, prioritize, and scope product requirements using a basic prioritization framework.
- Produce a one-page product brief that translates an idea into an actionable plan.

Software Systems and Architecture **2.00** **60.00**

The Software Systems and Architecture course builds the conceptual foundation every student needs before writing code. Students explore how computers process and store information, learn core data structures, develop a working understanding of how the internet functions, and study the design patterns and data modeling principles that shape how real software is architected. Rather than analyzing an arbitrary app, the course project has students take a product idea of their own and produce the architectural plan for it: the data model, the API surface, the client-server structure, and how information would need to move through the system end to end. The goal is to build the habit of designing before building. The course also introduces responsible data practices, asking students to weigh what their proposed product would collect, why it's needed, and what they're accountable for as its future developers.

Course Objectives:

- Demonstrate understanding of computer architecture, memory, storage, and operating system concepts.
- Apply common data structures to organize and manage information in a program.
- Explain how the internet works, including HTTP, DNS, and client-server communication, and how APIs move data between services.
- Apply software design patterns and data modeling principles to plan how a system should be structured.
- Produce an architectural plan for a product idea, including its data model, API surface, and system boundaries.
- Identify responsible data collection practices and describe the privacy implications of design decisions.

Software Development with AI	3.00	90.00

The Software Development with AI course is where students go from a plan to a working system. Students set up a professional development environment, write code in a general-purpose programming language, and use AI coding assistants and prompt engineering as core tools. Throughout the course, students critically evaluate the code AI produces, write tests to verify that AI-generated code actually does what it claims before it ships, and practice the judgment to know when to trust AI output and when to rewrite it. Ethical use is embedded in all projects through the principle that the developer is responsible for everything they ship, regardless of how it was written.

Course Objectives:

- Set up and navigate a professional software development environment including version control.
- Write and organize code using a general-purpose programming language.
- Use AI coding assistants and prompt engineering to accelerate development without sacrificing understanding.
- Critically evaluate AI-generated code for correctness, security, and appropriateness before shipping.
- Write tests to verify that AI-generated code behaves as intended before it ships.
- Build and deploy a scoped application that solves a defined, familiar problem using AI assistance.
- Design, build, and present a self-directed final project that turns a product idea into a working system.
- Articulate the developer's ethical responsibility for AI-assisted output.

AI Agents and Automation	4.00	120.00

The AI Agents and Automation course teaches students to design, build, and deploy agentic AI systems and automated workflows. Starting with no-code and low-code platforms, students progressively work with direct API integrations and agent frameworks to build systems that take real actions on behalf of users. The course covers how agents perceive inputs, reason through tasks, and produce outputs; how to design and sequence multi-step agentic workflows; and how to evaluate and debug agent behavior in production. Ethical considerations are central throughout: students examine the risks of autonomous systems, determine when human oversight is required, build appropriate guardrails, and reason through what accountability looks like when an agent makes a mistake that affects a real person.

Course Objectives:

- Build automated workflows using no-code and low-code platforms.
- Connect applications and services using APIs, webhooks, and event triggers.
- Explain how AI agents perceive inputs, reason through tasks, and produce outputs.
- Design and implement a multi-step agentic workflow using an agent framework.
- Evaluate and debug agent behavior using logging, testing, and prompt refinement.
- Identify failure modes of autonomous systems and implement appropriate guardrails.
- Apply human-in-the-loop design patterns in contexts where oversight is required.
- Articulate an accountability framework for agentic systems that affect real users.

Design Track Courses (Required Hours: 360, Credits: 12)

Design Track (Required Hours: 360, Credits: 12)	Credits	Hours
UX Foundations and Human-Centered Design	**2.00**	**60.00**

The UX Foundations and Human-Centered Design course introduces the fundamentals of human-centered design: how to translate a user problem into personas and journey maps, how information architecture shapes usability, and how to communicate a design idea through low-fidelity wireframes before any visual polish is applied. Starting from a defined product problem, students develop personas, a user journey map, and a wire-framed user flow.

Course Objectives:

- Explain the principles of human-centered design and why they shape product outcomes.
- Develop personas and user journey maps from a defined product problem.
- Apply information architecture principles to organize content and navigation.
- Produce low-fidelity wireframes that communicate a user flow clearly.
- Define a product problem and translate it into a UX foundation for further design work.

Visual Design and Prototyping **3.00** **90.00**

The Visual Design and Prototyping course takes students from wireframe to a working, high-fidelity prototype. Students learn core visual design principles (typography, color, layout, and hierarchy) and pair them with AI-powered design tools to generate, iterate, and refine interface concepts faster than manual design alone allows. The course also covers accessibility standards (WCAG basics) and responsive design across device sizes. From there, students build interactive, clickable prototypes that simulate real product behavior, turning static screens into something that feels and behaves like a working product. Students produce a polished, high-fidelity, interactive prototype for a wire-framed user flow, using AI tools as a drafting and iteration partner.

Course Objectives:

- Apply typography, color, layout, and visual hierarchy principles to interface design.
- Use AI-assisted design tools to generate and iterate on interface concepts.
- Design responsive layouts that adapt across common device sizes.
- Apply WCAG accessibility basics to color contrast, text sizing, and interactive elements.
- Critically evaluate AI-generated design output and revise it to meet usability and brand standards.
- Build interactive, clickable prototypes that simulate real product behavior.
- Produce a high-fidelity, interactive prototype for a wire-framed user flow.

Design Systems **2.00** **60.00**

The Design Systems course teaches students to design at scale rather than screen by screen. Students learn to identify what belongs in a shared design system versus what stays a one-off decision, then build that system out: reusable components, design tokens, and documented patterns that keep a product consistent as it grows. The course also covers how to prepare a design system for handoff to engineering, with clear specs, usage guidelines, and annotations a development team can act on without guessing. Students build a small component library from a high-fidelity interface, complete with documentation ready for handoff.

Course Objectives:

- Explain the purpose and structure of a design system, including components, tokens, and patterns.
- Identify which interface elements belong in a shared system versus a one-off design.
- Build a reusable component library from an existing product interface.
- Document component usage guidelines and design specifications.
- Prepare a design system for handoff to engineering teams.

User Research and Usability Testing **3.00** **90.00**

The User Research and Usability Testing course teaches students to validate design decisions with real users instead of assumptions. Students learn common research methods (interviews, surveys, and moderated usability testing) and how to choose the right method for a given question. The course covers how to write a research plan, recruit representative participants, and run a testing session without leading the user. Students run testing across multiple rounds against an interactive prototype and a set of interface components, revising their materials between rounds based on what they learn. Students also learn how AI tools can accelerate analysis of qualitative data (interview transcripts, survey responses) while examining where that automation introduces bias or loses nuance. The course closes with a findings report that translates research into prioritized, actionable design recommendations.

Course Objectives:

- Select an appropriate research method for a given design question.
- Write a research plan and recruit representative participants.
- Conduct a moderated usability test without leading or biasing the participant.
- Test an interactive prototype and design system components with real users across multiple rounds.
- Synthesize qualitative research data into clear, prioritized findings.
- Use AI tools to accelerate analysis of research data while identifying where automation introduces bias.
- Produce a usability findings report with actionable recommendations for the prototype and design system.

Product Management, Roadmapping, and Agile Practice	2.00	60.00

The Product Management, Roadmapping, and Agile Practice course zooms out from individual screens to the practice of managing a product over time. Students learn how product managers build and prioritize roadmaps, balance stakeholder input against user needs, and translate design and research work into a backlog a development team can execute. The course covers agile ceremonies (stand ups, sprint planning, retrospectives) and how designers operate inside them, plus how to pitch a product vision to stakeholders. Students close the course by assembling their own design and research work (personas, prototype, design system, and usability findings) into a product roadmap and a final stakeholder presentation.

Course Objectives:

- Explain how product roadmaps are built, prioritized, and communicated to stakeholders.
- Translate design and research work into a backlog of actionable items for a development team.
- Participate in agile ceremonies and describe the designer's role within them.
- Balance competing stakeholder priorities against user needs in product decisions.
- Assemble personas, design work, and usability findings into a cohesive product roadmap and stakeholder presentation.

Web Track Courses (Required Hours: 360, Credits: 12)

Web Track (Required Hours: 360, Credits: 12)	Credits	Hours
Modern Front-End, AI-Accelerated	**2.00**	**60.00**

The Modern Front-End, AI-Accelerated course teaches students to build the browser platform before adopting a framework. Building on the shared core's programming fundamentals, HTTP, Git workflow, and AI-assisted development practices, students create semantic, responsive, and accessible interfaces with HTML, CSS, JavaScript, and TypeScript, and connect those interfaces to live web services. Students use browser developer tools to debug, audit, and improve real application behavior, with checkpoints requiring them to critique, correct, test, and justify AI-generated code.

Course Objectives:

- Build semantic, accessible HTML and implement responsive layouts using the box model, Flexbox, Grid, media and container queries, and reusable design tokens.
- Write modular, typed client-side code in JavaScript and TypeScript using functions, objects, arrays, modules, events, promises, and async/await.
- Connect pages to live data from web services and handle loading, empty, success, and error states without losing a user's work.
- Build accessible, validated forms while distinguishing client-side convenience validation from server-side security controls.
- Debug client-side applications using browser developer tools and write unit tests for deterministic logic.
- Evaluate AI-assisted code against product requirements, tests, accessibility, security, and licensing expectations.

React and Component Architecture	3.00	90.00

The React and Component Architecture course has students use React and TypeScript to transform product requirements into multi-route web applications. Students design reusable component systems, manage local and shared state, build accessible forms, implement client-side navigation and authentication workflows, and connect applications to live APIs and external services. They test applications from the user's perspective, measure and improve performance, safely refactor existing code, and apply core AI competencies by building an AI-powered interface with streaming responses and meaningful user controls.

Course Objectives:

- Design maintainable component architectures by breaking product requirements into organized, reusable React components.
- Manage application state and behavior using React hooks, predictable data flow, Context, and appropriate state-management patterns.
- Build complete multi-route applications with client-side navigation, authentication workflows, protected routes, and accessible forms.
- Integrate live APIs and external services while handling loading, empty, success, optimistic updates, validation, and error states.
- Write user-focused tests, measure application performance, and implement evidence-based improvements.
- Review code, communicate architectural decisions, and safely refactor duplicated or difficult-to-maintain code.
- Develop responsible AI-powered features that support streaming responses, cancellation, error recovery, and meaningful user control.

Back-End and API Engineering	3.00	90.00

The Back-End and API Engineering course has students design and implement secure, production-oriented server applications using Node.js, TypeScript, Express, and PostgreSQL. Students model relational data, develop documented REST APIs, validate untrusted input, implement authentication and authorization, and diagnose failures through structured logging and automated tests. Students then compare that relational architecture with Firebase and Firestore, and securely integrate an AI service behind a protected server boundary with credential protection, validation, rate and cost controls, and user-safe fallback behavior.

Course Objectives:

- Explain the Node.js runtime environment, including asynchronous I/O, package management, and environment-based configuration.
- Design, implement, and document RESTful APIs in TypeScript with typed routes, middleware, status codes, pagination, and an OpenAPI contract.
- Model relational domains in PostgreSQL and write SQL for CRUD operations, joins, aggregates, and transactions; apply keys, constraints, normalization, and migrations.
- Implement layered application security, including input validation, secure credential storage, session or token authentication, and least-privilege authorization.
- Write unit and integration tests covering success, validation, authentication, authorization, and failure paths, and implement structured logging and health checks.
- Build a Firestore feature with security rules, justify relational vs. document-database vs. backend-as-a-service tradeoffs, and integrate an AI service behind a secured boundary with rate, cost, and fallback controls.

Cloud, DevOps and Web Security	2.00	60.00

The Cloud, DevOps and Web Security course has students deploy and operate a full-stack application in a repeatable, production-like cloud environment. They apply Linux and networking fundamentals, containerize multi-service applications with Docker, configure a focused AWS deployment architecture, build automated CI/CD pipelines with GitHub Actions, and implement monitoring and recovery practices. Security, accessibility, performance, search discoverability, AI-service reliability, and cost awareness are treated as measurable release requirements rather than afterthoughts.

Course Objectives:

- Diagnose application delivery problems using Linux command-line, process, file-permission, environment, port, DNS, and HTTP/TLS concepts.
- Construct production-oriented container images and deploy a containerized web application and database with Docker and Docker Compose across development, test, and production configurations.
- Configure secure AWS environments applying identity and access management, least-privilege principles, domains, DNS, HTTPS, and secrets management.
- Build an automated CI/CD pipeline with GitHub Actions that installs, lints, tests, scans, builds, and deploys the application, with health checks, logging, and a documented rollback procedure.
- Identify and remediate representative OWASP Top 10 security risks, including access control, injection, authentication, and supply-chain failures.
- Evaluate release readiness against a production gate covering accessibility, Core Web Vitals, caching, SEO, error handling, and AI-integration monitoring.

Capstone and Career Launch	2.00	60.00

The Capstone and Career Launch course has teams of two synthesize the competencies of the full program by delivering and defending a deployed full-stack product while maintaining documented evidence of individual contribution. Students validate the problem, refine requirements, define acceptance criteria, and determine how AI contributes to the product's value, managing work through issues, pull requests, and architecture decisions. Career preparation converts the capstone into an employer-ready portfolio and develops the code-review, debugging, and interviewing skills expected of entry-level developers.

Course Objectives:

- Convert an ambiguous stakeholder problem into a bounded product brief with a prioritized backlog, acceptance criteria, data model, and delivery plan.
- Justify and document proportionate architecture decisions and execute them through a professional team workflow of small issues, branches, and code review.
- Construct an accessible and responsive React and TypeScript client integrated with a documented, secure, data-backed API and a bounded AI capability, delivered as complete vertical slices.
- Validate the product through unit, component, integration, and end-to-end tests, and deploy through a CI/CD pipeline with verified security, accessibility, and recovery procedures.
- Prioritize defects and technical debt by user impact, risk, effort, and release timing, and defend individual code contributions, including AI-assisted work.
- Present an employer-ready professional portfolio and demonstrate entry-level interview competencies through live coding, code review, and behavioral examples.

iOS Track Courses (Required Hours: 360, Credits: 12)

iOS Track (Required Hours: 360, Credits: 12)	Credits	Hours

Swift Fundamentals, AI-Accelerated	**2.00**	**60.00**

The Swift Fundamentals, AI-Accelerated course teaches students to read and write idiomatic Swift, the language underneath every iOS app, with AI tools accelerating practice and review. The course closes the loop from language mechanics to working software: persisting simple data, pulling in packages, writing tests, and shipping a small command-line project.

Course Objectives:

- Write idiomatic Swift using optionals, collections, closures, and value vs. reference types.
- Model app state and data with enums, protocols, extensions, and generics.
- Handle failures explicitly with throws, do/catch, and the Result type.
- Encode and decode real-world JSON with Codable, and persist lightweight data with UserDefaults and the file system.
- Debug with breakpoints and LLDB, manage dependencies with Swift Package Manager, and write unit tests.
- Scope, build, test, and refactor a complete Swift command-line app, using AI assistance with human review.

SwiftUI and App Architecture	**3.00**	**90.00**

The SwiftUI and App Architecture course teaches students to build real, multi-screen iOS interfaces with SwiftUI. Students progress from layout and state fundamentals through navigation, animation, and adaptive design to the skills that separate a demo from a maintainable app: framework integration, testing, accessibility, and architecture.

Course Objectives:

- Compose adaptive layouts with stacks, grids, ScrollView, and GeometryReader that work across device sizes and orientations.
- Manage state correctly with @State, @Binding, @Observable, and Environment, maintaining a single source of truth.
- Implement multi-screen navigation with NavigationStack, tabs, sheets, and deep links.
- Design for the full data lifecycle (loading, error, and empty states) with resilient, recoverable UI.
- Find, evaluate, and integrate first- and third-party frameworks by working from their documentation.
- Test view models and UI flows (Swift Testing, XCUITest) and meet accessibility expectations (VoiceOver, Dynamic Type).
- Organize a growing codebase (MVVM, modularization) and defend architecture decisions in review.

Networking, Persistence and Cloud Backends	**3.00**	**90.00**

The Networking, Persistence and Cloud Backends course builds the data layer end to end. Students build the memory and concurrency foundations that networking depends on, then consume real APIs through a typed networking layer. They persist data locally with SwiftData, learn how relational databases are structured, and stand up cloud backends with Firebase. By the end, they can choose and justify the right data architecture for a given app.

Course Objectives:

- Manage memory and concurrency safely: ARC and retain cycles, async/await, actors, and main-thread rules.
- Build a reusable networking layer on URLSession that decodes complex JSON and handles errors, retries, and offline states.
- Design relational database structure: tables, keys, relationships, normalization, and joins.
- Persist and query structured data with SwiftData, and read legacy Core Data code.
- Implement authentication (tokens, OAuth, Sign in with Apple) and protect secrets with Keychain and build configurations.
- Build a Firebase/Firestore backend: data modeling, CRUD, auth, security rules, cloud functions, and push notifications.
- Compare local, BaaS, CloudKit, and custom-API options and justify a data-layer choice for a real project.

App Store, DevOps and App Marketing **2.00** **60.00**

The App Store, DevOps and App Marketing course covers everything between an app that works and an app that people are using. Students take a finished build through signing, TestFlight, and App Store review; automate testing and releases with CI/CD; monitor what happens after launch; and market the app with a brand, a website, and a social presence that feed into App Store optimization.

Course Objectives:

- Take an app from Xcode build through code signing, TestFlight beta testing, and App Store submission.
- Design and run a CI/CD pipeline (Xcode Cloud, fastlane, GitHub Actions) with automated test suites.
- Monitor release health with crash reporting, analytics, and alerting, and respond to regressions.
- Audit and optimize performance, app size, and accessibility before release.
- Create an app brand identity, build a marketing landing page, and plan a social media launch strategy.
- Optimize an App Store listing (ASO) and plan pricing, in-app purchases, and phased rollouts.

Capstone and Career Launch **2.00** **60.00**

The Capstone and Career Launch course has teams of two scope, build, and ship a complete iOS app through sprints, integration points, code reviews, and quality passes to a TestFlight release and public demo day. The final stretch of the course converts that work into career momentum: portfolio, interviews, and offers.

Course Objectives:

- Scope a buildable product, plan its architecture, and divide work across a two-person team.
- Ship working software in sprints, integrating early, triaging scope, and paying down technical debt.
- Run pre-release quality passes (testing, performance, accessibility, security) and distribute via TestFlight.
- Present and demo technical work to technical and non-technical audiences.
- Produce job-search assets: resume, portfolio, GitHub profile, and personal brand.
- Perform in technical and behavioral interviews, and evaluate and negotiate job offers.